/no_idea — техдок и спринты

Oct 25, 2026 · @Bodya

Бот /no_idea сидит в рабочем ТГ-чате и превращает обсуждение темы в живой канвас на сайте. На закрытии темы он присылает в чат картинку канваса и короткое саммари. Техдок берёт пайплайн из прошлой версии и меняет только вход и выход: вместо веб-чата — Telegram, вместо «Собрать план» — итог темы.

Обзор

Проблема. Тему обсуждают в сотне сообщений рабочего чата. Через час никто не помнит, на чём сошлись, а тому, кто выпал, приходится листать всё. Отдельный сервис для обсуждений никто не откроет: люди уже сидят в ТГ.

Решение. Бот слушает чат, пока открыта тема, и ведёт канвас — тезисный конспект в виде дерева. В чат пишет редко: когда мнения расходятся, когда что-то непонятно или когда его спросили.

Что делает бот:

- ведёт канвас: вытаскивает тезисы, раскладывает по блокам, сжимает повторы;
- сводит одну цель, сказанную разными словами, и помечает настоящий конфликт с вопросом в чат;
- при смене ограничения подсвечивает, что придётся пересмотреть;
- отвечает на вопросы по канвасу («на чём мы сошлись по бюджету?»);
- на закрытии присылает PNG канваса + саммари до 5 строк + файл .md.

Подача: тема 3 «Второй мозг». Бот работает там, где команда уже общается, и применяет накопленный контекст в нужный момент. Репозиторий под MIT.

MVP (к 15:45):

- Команды start / stop / export / ask по имени темы, одна открытая тема на чат.
- Живой канвас на сайте по ссылке от бота, обновление через 2–3 с после сообщения.
- Сведение одной цели, конфликт с вопросом в чат, последствия смены ограничения.
- Картинка канваса по команде и на закрытии + саммари + экспорт .md.
- Вопросы боту по канвасу.

Убрано из прошлой версии: веб-чат как основной вход (остаётся только replay для страховки), режим тезис-плана и Planner, ручная правка тезисов на сайте, регистрация.

Что достать руками до старта

Всё ниже занимает минут 20, и без этого Claude не сможет ни протестировать бота, ни вызвать LLM. Ключи кладёшь в `.env` на ноутбуке, не в чат и не в git.

- [ ] **Токен бота.** В Telegram открой @BotFather → `/newbot` → имя «no idea» → username, например `no_idea_hack_bot` → скопируй токен в `TG_BOT_TOKEN`. Если бот фуллстека уже есть, бери его токен.
- [ ] **Выключить privacy mode.** В @BotFather: `/setprivacy` → бот → **Disable**. Иначе в группе бот видит только команды. После смены удали бота из групп и добавь заново, иначе настройка не применится.
- [ ] **Разрешить группы.** `/setjoingroups` → Enable (обычно уже включено).
- [ ] **Две группы.** «no_idea dev» для разработки и «no_idea demo» для защиты. Добавь бота в обе, сделай админом. В demo — четыре аккаунта участников.
- [ ] **Ключ LLM.** Проще всего один ключ OpenRouter (openrouter.ai → Keys) или Anthropic API (console.anthropic.com). Положи $10–20 на счёт, на день хватит. В `.env`: `LLM_BASE_URL`, `LLM_API_KEY`, `FAST_MODEL` (класса Claude Haiku 5.5), `STRONG_MODEL` (класса Claude Sonnet 5.5).
- [ ] **Репозиторий на GitHub** под MIT, с кодом бота фуллстека и готовым фронтом канваса, если он есть. Дай Claude доступ к нему.
- [ ] **Ноутбук-сервер.** Python 3.11+, Node 20, `pip install playwright && playwright install chromium` (для картинки канваса). Бот работает через long polling, публичный адрес для него не нужен.
- [ ] **Публичная ссылка на сайт** (чтобы канвас открывался с телефонов): `cloudflared tunnel --url http://localhost:8000`, без аккаунта. Полученный адрес — в `PUBLIC_BASE_URL`. План Б: ngrok.
- [ ] **Мобильная точка доступа** на случай, если Wi-Fi площадки режет Telegram.

Жизненный цикл темы и команды

Все команды адресуют тему по имени. В чате открыта не больше одной темы, закрытых может быть сколько угодно. Пока тема открыта, все текстовые сообщения чата идут в неё. До start и после stop бот ничего не читает и не хранит.

1. **Открытие.** `/no_idea start Онбординг` → «Слушаю тему «Онбординг». Канвас: <ссылка>». Если тема с таким именем уже была, она переоткрывается: канвас продолжается с того же места, ссылка та же. Отдельный reopen не нужен.
2. **Обсуждение.** Люди пишут как обычно. Канвас на сайте живёт, бот изредка вмешивается и отвечает на вопросы.
3. **Закрытие.** `/no_idea stop Онбординг` → PNG канваса с подписью-саммари до 5 строк. Канвас на сайте замораживается с плашкой «Тема закрыта».
4. **После.** По закрытой теме можно в любой момент сделать export и ask.

Команда	Что делает	Ошибки
`/no_idea start <имя>`	Открыть новую тему или переоткрыть старую; прислать ссылку на канвас	Уже открыта другая тема → «Сначала stop «…»»
`/no_idea stop <имя>`	Закрыть тему: PNG + саммари	Тема не открыта
`/no_idea export <имя>`	Актуальный PNG канваса + файл .md + ссылка; работает и для открытой, и для закрытой	Темы нет → список тем чата
`/no_idea ask <имя> <вопрос>`	Ответ по канвасу и истории темы, до 4 предложений	Темы нет → список тем чата
`/no_idea list`	Темы чата: имя, статус, ссылка	—
`/no_idea` или `/no_idea help`	Список команд и текущая тема	—

Правила разбора имени:

- Имя из нескольких слов — в кавычках: `/no_idea ask "Редизайн онбординга" что решили по срокам?`. Без кавычек имя — первое слово.
- Для stop, export и ask имя можно не писать, если тема открыта: берём текущую. Для ask по текущей теме также работает реплай на бота.
- Имена сравниваем без учёта регистра и пробелов по краям; если точного совпадения нет, а по префиксу подходит ровно одна тема — берём её.
- В группах Telegram может прислать `/no_idea@no_idea_hack_bot` — парсер срезает `@…`.

Команды и вопросы боту на канвас не попадают. Фото, стикеры и голосовые игнорируем, у фото берём только подпись. Редактирование сообщений в MVP игнорируем.

Когда бот пишет в чат

Бот сам пишет только тогда, когда нужен ответ людей. Всё остальное видно на канвасе. В рабочем чате бот, который болтает, замьютят за час.

Ситуация	Что пишет бот	Пример
Настоящий конфликт	Называет конфликт, 2–3 варианта и цену каждого, спрашивает, что важнее	«Скидка 20% бьёт по марже, а бюджет ограничен. Что важнее: вернуть клиентов или маржа?»
Смена ограничения с последствиями	Что изменилось и какие договорённости пересмотреть	«Срок стал 2 недели. Наборы под вопросом: начать с оформления?»
Спорят о разном (недопонимание)	Один уточняющий вопрос авторам	«Лендинг и простая главная — это одно и то же?»
Компромисс для совместимых целей	Одна фраза: что добавил на канвас и чьи цели учёл	«Персональные предложения можно сделать простыми правилами, без ML. Добавил на канвас.»
Одна цель, разные слова	Молчит, тост только на канвасе	—
Вопрос к боту	Ответ реплаем на вопрос, до 4 предложений, со ссылкой на тезисы	«По бюджету сошлись: скидка не больше 5% и только на наборы (решил Руководитель).»

Правила тишины:

- Сам по себе — не чаще одного сообщения в 60 с (`BOT_COOLDOWN_S`). На вопросы и команды ограничение не действует.
- До 3 предложений, обращение по именам из Telegram, без комплиментов и канцелярита.

- Вопрос по конфликту — реплаем на сообщение, которое его вызвало. Ответ людей реплаем на бота сразу разрешает конфликт (триггер RESOLVE).
- В каждом сообщении бота — кнопка «Показать на канвасе» (ссылка с `#N7`, сайт подсвечивает узел).

Канвас и сайт

Канвас остаётся из прошлого техдока: фиксированный каркас из 7 блоков, в каждом дерево тезисов глубиной 2, не больше 6 тезисов на блок.

Блок	Что туда попадает
Суть	Одна фраза о том, на чём сейчас стоит команда
Проблема	Боль, ради которой всё затевается
Для кого	Аудитория и сегменты
Решение	Предложения; дети — способы и детали
Ограничения	Бюджет, сроки, люди, технические пределы
Риски	Опасения и возражения
Открытые вопросы	Неразрешённое, в том числе вопросы по конфликтам

Статусы тезиса те же: new (подсветка 3 с), active (имена авторов, «+N»), disputed (красная рамка + ребро), needs_review (жёлтая рамка + пунктир к ограничению), rejected (серый, зачёркнут, причина в тултипе). Флага pinned в MVP нет: ручной правки на сайте не делаем.

Сайт — только просмотр, без регистрации. Доступ по ссылке со случайным токеном: `{PUBLIC_BASE_URL}/t/{web_token}`. Ссылка известна только участникам чата, этого хватает на хакатон. Telegram Login — растяжка на потом.

На странице: сверху название темы, статус (открыта / закрыта) и индикатор «бот читает / думает»; под ним канвас на всю ширину; справа узкая лента последних сообщений из ТГ. Клик по тезису показывает цитаты исходных реплик с авторами — ответ на «а если ИИ наврал?».

PNG для бота делается на сервере: Playwright открывает `/t/{web_token}?print=1` (без ленты сообщений и анимаций, все блоки развёрнуты, ширина 1600 px) и делает скриншот. Так картинка совпадает с сайтом, и не нужен второй

рендер. Браузер держим открытым всё время, снимок занимает ~1 с.

Архитектура

Один Python-процесс держит бота, API сайта, состояние тем и ML-пайплайн. Главный принцип из прошлого техдока остаётся: LLM никогда не перегенерирует канвас целиком, она возвращает операции, а Reducer на Python их проверяет и применяет.

Стек

Слой	Выбор	Почему
Бот	aiogram 3, long polling	Асинхронный, живёт в одном event loop с FastAPI; не нужен публичный адрес
Бэкенд	Python 3.11+, FastAPI, uvicorn	REST + WebSocket для сайта
Схемы	Pydantic v2	Одни модели дают JSON-схему для LLM и валидацию
LLM	openai SDK с base_url на OpenRouter или Anthropic-совместимый endpoint	Модель меняется одной переменной
Хранение	Состояние в памяти + журнал JSONL на тему	Ноль инфраструктуры, рестарт проигрыванием журнала
Кэш LLM	Файловый, по хэшу запроса; off / record / replay	Демо воспроизводимо
Сайт	React (Vite): CSS grid + react-xarrows + framer-motion; или готовый фронт канваса фуллстека	Сборку раздаёт тот же FastAPI
PNG	Playwright (Chromium), скриншот print-режима страницы	Картинка = сайт, один рендер

Путь одного сообщения

1. В группе пишут сообщение. aiogram получает его сразу (не раз в минуту).

2. Роутер: команда `/no_idea …` → обработчик команд; реплай на бота → ask или ответ на конфликт; иначе, если в чате есть открытая тема, → `TopicRoom.on_message`. Иначе игнор.
3. `on_message` выдаёт seq, пишет в журнал, шлёт сообщение на сайт по WS, кладёт в очередь темы.
4. Воркер темы ждёт debounce 1,5 с и забирает пачку. На тему всегда один обработчик.
5. Extractor → Reducer → canvas_patch на сайт. Тезис появляется через 2–3 с.
6. Trigger policy решает, нужен ли Mediator, Impact или Consolidator. Если нужен — его операции снова через Reducer, а `chat_message` уходит в группу через `bot.send_message` (с учётом cooldown).

API сайта

Метод	Путь	Что делает
GET	/t/{token}	Страница канваса (SPA); `?print=1` — режим для скриншота, `#N7` — подсветка узла
GET	/api/topics/{token}	Полное состояние: тема, сообщения, канвас
GET	/api/topics/{token}/export.md	Markdown one-pager шаблоном, без LLM
WS	/ws/topics/{token}	snapshot → message, canvas_patch, toast, agent_status, topic_status
POST	/api/topics/{token}/replay	Проиграть scenarios/*.json как будто из ТГ (страховка и тесты)
GET	/debug/llm	Последние 20 LLM-вызовов

Структура репозитория

```
backend/app/
  main.py        # FastAPI + запуск aiogram в том же loop
  bot.py         # роутер команд, парсер имени, отправка фото/файлов,
cooldown
  topics.py      # TopicRoom: состояние, очередь, воркер, broadcast; реестр
тем по chat_id
  models.py      # Pydantic: тема, канвас, операции, ответы LLM
  reducer.py     # применение и валидация операций, выдача id
  guards.py      # запрет принудительного слияния, глубина
  triggers.py    # trigger policy
  impact.py      # граф зависимостей
  llm.py         # клиент, structured output, ретраи, кэш, лог
  pipeline.py    # extract -> reduce -> trigger -> mediate/impact/consolidate
  render.py      # канвас -> текст для промпта; канвас -> markdown
  snapshot.py    # Playwright: PNG канваса
  prompts/       # extractor, consolidator, mediator, impact, summarizer,
answerer
  scenarios/     # demo_shop.json, teammates.json, coffee.json
  data/          # jsonl-журналы, llm_cache/, llm_log.jsonl
frontend/        # страница канваса
.env.example     # TG_BOT_TOKEN, LLM_*, FAST_MODEL, STRONG_MODEL,
PUBLIC_BASE_URL, CACHE_MODE
```

Модель данных и роли LLM

Канвас, операции, Reducer, guards, trigger policy, impact-анализ и промпты Extractor / Consolidator / Mediator / Impact берём **без изменений** из прошлого техдока (разделы «Модель данных», «ML-пайплайн», «Промпты»). Ниже только то, что меняется.

Изменения в models.py

```
class Message(BaseModel):
    seq: int; ts: float; text: str
    author: str                    # имя из Telegram (first_name или
@username)
    tg_user_id: int | None = None
    tg_message_id: int | None = None
    reply_to_seq: int | None = None  # если это реплай — контекст для
Extractor
    is_agent: bool = False
```

```
class Topic(BaseModel):                # бывший Idea
    id: str                            # T3
    chat_id: int
    name: str                          # как написали в start
    key: str                           # name.casefold().strip() — для поиска
    web_token: str                     # secrets.token_urlsafe(12)
    status: Literal["open", "closed"] = "open"
    opened_by: str; opened_at: float
    closed_at: float | None = None
    summary: str | None = None         # последнее саммари
    messages: list[Message] = []
    canvas: Canvas = Canvas()
    counters: dict[str, int] = {}

class ChatState(BaseModel):
    chat_id: int
    open_topic_id: str | None = None
    topics: dict[str, str] = {}        # key -> topic_id
    last_bot_message_ts: float = 0     # для cooldown
```

Из Canvas убираем `plan` и `mode`, из Node — `pinned`. У каждого сообщения бота храним `tg_message_id` и связанный conflict_id, чтобы реплай на вопрос бота сразу попадал в RESOLVE.

Шесть LLM-ролей

Роль	Когда	Модель	Что возвращает
Extractor	На каждую пачку (debounce 1,5 с)	Быстрая, до 2 с	Операции над канвасом, как раньше
Consolidator	Каждые 8 сообщений, блок > 6 тезисов, перед stop	Сильная, в фоне	Слияния, переносы, новая «Суть»
Mediator	Конфликт, одна цель, ответ на конфликт	Сильная, до 6 с	Вердикт + операции + сообщение в ТГ
Impact	Изменилось ограничение с зависимыми узлами	Сильная, до 6 с	keep / revise / drop + сообщение в ТГ
Summarizer (новая)	stop и export	Сильная, до 6 с	Саммари до 5 строк для подписи к PNG

Роль	Когда	Модель	Что возвращает
Answerer (новая)	ask / реплай на бота	Быстрая, до 3 с	Ответ до 4 предложений + id узлов

Planner убран. Итог темы — канвас + саммари.

Промпт Summarizer

```
Ты пишешь итог обсуждения для рабочего чата по его канвасу.
До 5 строк, каждая до 15 слов, без вводных:
1) суть — что решили делать;
2-3) главные договорённости (с именами, если кто-то взял задачу);
4) от чего отказались и почему, если было;
5) открытые вопросы — их число и самый важный.
Только то, что есть на канвасе. Узлы REJECTED — только в строке 4.
Узлы DISPUTED и NEEDS_REVIEW — не договорённости, а открытые вопросы.
```

Подпись к фото в Telegram — до 1024 символов. Код обрезает до 1000 и дописывает ссылку на канвас.

Промпт Answerer

```
Тебе задали вопрос про обсуждение в чате. Отвечай ТОЛЬКО по канвасу
и сообщениям ниже. До 4 предложений. Называй, кто что сказал или решил.
Если на канвасе ответа нет — так и скажи и предложи спросить команду.
Верни answer и node_ids — узлы, на которых основан ответ.
```

Во все промпты канвас идёт одним компактным рендером (`render.canvas_compact`), температура 0–0,2, structured output через JSON-схему из Pydantic.

Демо и тесты в Telegram

Демо — живая группа «no_idea demo»: четверо пишут с телефонов, на проекторе слева Telegram Desktop, справа канвас. Сценарий — «Поднять продажи интернет-магазина» из прошлого техдока, обёрнутый командами.

#	Шаг	Что видит жюри
0	`/no_idea start Продажи`	Бот присылает ссылку, на проекторе пустой каркас из 7 блоков

#	Шаг	Что видит жюри
1–2	Руководитель и Маркетолог: цель +15%, «люди уходят», персональные предложения	Заполняются Суть, Ограничения, Проблема, Решение
3	Дизайнер: «сократим оформление»	Два тезиса сливаются в одну цель, тост на канвасе, бот молчит
4	Разработчик: «нас двое и месяц»	Ограничение + деталь-компромисс, бот пишет одну фразу
5	Маркетолог: «рассылка со скидкой 20%»	Красное ребро, бот реплаем спрашивает, что важнее
6	Руководитель реплаем на бота: «Маржа важнее, скидка максимум 5% на наборы»	«Рассылка 20%» зачёркнута, новая деталь
7	Руководитель: «бюджет дали, но срок 2 недели»	Карточки желтеют, бот пишет, что пересмотреть
8	`/no_idea ask Продажи что решили по скидке?`	Ответ с именами и кнопкой «Показать на канвасе»
9	`/no_idea stop Продажи`	В чате PNG канваса + саммари. Финал: `/no_idea export Продажи` → .md

Реплики игроков лежат в заметках на телефонах и вставляются копипастом, до символа как на репетиции — тогда сработает replay-кэш. Пауза между репликами больше debounce (3–4 с).

Тестовые чаты

Три сценария из прошлого техдока (demo_shop, teammates, coffee) остаются как есть с тем же чек-листом. Проигрываются через `POST /replay` без Telegram: так промпты проверяются за минуту, а не руками в группе.

Чек-лист ручного теста в Telegram

- [ ] Бот видит обычные сообщения в группе (privacy mode выключен, бот передобавлен)
- [ ] Все команды работают в виде `/no_idea` и `/no_idea@no_idea_hack_bot`; имя в кавычках и без

- [ ] start при открытой теме даёт понятную ошибку; start по старому имени переоткрывает ту же ссылку
- [ ] После stop сообщения не попадают на канвас
- [ ] PNG приходит за < 5 с, читается на телефоне, подпись не длиннее 1024 символов
- [ ] Реплай на вопрос бота по конфликту разрешает конфликт
- [ ] Cooldown: бот не пишет сам чаще раза в 60 с
- [ ] Ссылка на канвас открывается с телефона через туннель
- [ ] Рестарт сервера не теряет темы (проигрывание журнала)
- [ ] Два чата одновременно не путают темы

Риски и страховка

Риск	Что делаем
Wi-Fi площадки режет Telegram	Ноутбук-сервер через точку доступа с телефона; план Б — `POST /replay` играет сценарий без ТГ, канвас на проекторе
Бот не видит сообщения в группе	Privacy mode выключен + бот передобавлен + админ; проверяем первым делом в спринте 0
На защите модель отвечает иначе	CACHE_MODE=replay + реплики копипастом из заметок. Ключ кэша не должен содержать ts, tg_message_id и токены — только тексты, имена и канвас
Бот болтает и раздражает	Cooldown 60 с, в чат только конфликт / ограничение / уточнение / ответ
Playwright падает или долго грузится	Браузер стартует один раз при запуске; таймаут 10 с → бот шлёт саммари текстом + ссылку
Агент склеил конфликт	Guards в коде, как в прошлом техдоке; тестовые чаты ловят регрессии
Конспект переврал	Клик по тезису на сайте показывает исходные реплики
Ничего не работает	Запасное видео в 17:00: ТГ + канвас рядом

Спринты

Код пишет Claude, ты даёшь задание и в конце каждого спринта 15 минут тестируешь в Telegram. Каждый блок «Задание для Claude» копируется целиком; в первом задании дай Claude ссылку на этот док.

#	Время	Цель	Твой тест в ТГ в конце
0	11:30–12:15	Скелет: бот видит чат и понимает команды	start / stop / list отвечают, сообщения видны на сайте сразу
1	12:15–13:30	Живой канвас из ТГ	Реплики 1–4 демо собирают канвас за 2–3 с
2	13:30–14:45	Бот вмешивается и отвечает	Реплики 1–8: конфликт, реплай-разрешение, последствия, ask
3	14:45–15:45	Итог темы. **MVP-фриз в 15:45**	stop → PNG + саммари, export → PNG + .md, переоткрытие
4	15:45–17:00	Надёжность и ТГ-тесты	Весь чек-лист ручного теста + репетиция в record
5	17:00– защита	Страховка и питч	Видео записано, replay прогнан

Спринт 0 · 11:30–12:15 · Скелет и бот

Зависит от: токен бота, privacy mode, репозиторий (раздел «Что достать руками»). Оценка: ~45 мин.

```
Задание для Claude — спринт 0.
Прочитай техдок /no_idea (ссылка) и старый код бота в репозитории.
1. Создай структуру репозитория из раздела «Архитектура», LICENSE (MIT),
   .env.example, requirements.txt (fastapi, uvicorn, aiogram 3, pydantic 2,
openai,
   playwright), README с запуском одной командой.
2. models.py по разделу «Модель данных» (Canvas/Node/Conflict — из старого
техдока
   без plan, mode, pinned; Topic, ChatState, Message — из нового).
3. main.py: FastAPI + aiogram long polling в одном event loop.
4. bot.py: команда /no_idea с подкомандами start/stop/export/ask/list/help по
   таблице команд и правилам разбора имени (кавычки, @botname, текущая тема
   по умолчанию, префикс). export и ask пока отвечают заглушкой. Парсер —
   чистая функция с юнит-тестами.
5. topics.py: реестр тем по chat_id, одна открытая тема, start по старому
имени
   переоткрывает. TopicRoom.on_message: seq, журнал data/{topic_id}.jsonl,
broadcast
   по WS. Все текстовые сообщения чата с открытой темой идут туда сразу
   (никакого опроса раз в минуту); команды и медиа без подписи — нет.
6. GET /t/{token} — простая страница с лентой сообщений из WS и пустым
каркасом
   из 7 блоков; GET /api/topics/{token}; WS /ws/topics/{token} со snapshot.
Запусти тесты парсера. Не коммить .env.
```

Готово, когда: в «no_idea dev» `/no_idea start Тест` присылает ссылку, сообщения сразу появляются на странице, `/no_idea stop` останавливает поток.

Спринт 1 · 12:15–13:30 · Живой канвас

Зависит от: спринт 0, ключ LLM. Оценка: ~1 ч 15 мин.

```
Задание для Claude — спринт 1.
1. llm.py по старому техдоку: structured(), один ретрай с текстом ошибки, лог
в
   data/llm_log.jsonl, кэш off/record/replay. Ключ кэша — только модель +
промпт +
   тексты/имена/канвас, без ts и telegram id. Сначала проверь одним запросом,
   что провайдер из .env поддерживает json_schema; иначе tool calling.
2. render.canvas_compact и extractor_input; промпт prompts/extractor.md +
ExtractResult —
   дословно из старого техдока, автор = имя из Telegram, reply_to_seq в
контексте.
3. reducer.apply_extract: выдача id, parent_ref, отброс несуществующих id и
глубины > 2,
   new -> active, сигналы для триггеров. Юнит-тесты.
4. Воркер TopicRoom: debounce 1,5 с, один обработчик на тему, process_batch
только с
   Extractor, canvas_patch (upsert_node/remove_node/upsert_conflict) и
agent_status по WS.
5. Страница канваса: CSS grid 7 блоков, карточки с деталями, имена авторов,
«+N»,
   подсветка new 3 с, framer-motion layout; при пропуске version —
перезапрос.
   Если в репозитории есть готовый фронт канваса — используй его.
6. Клик по тезису — попап с цитатами source_msgs и авторами.
7. scenarios/demo_shop.json (реплики из старого техдока) + POST
/api/topics/{token}/replay.
Прогони replay demo_shop до реплики 4 и покажи канвас в компактном рендере.
```

Готово, когда: в ТГ реплики 1–4 демо заполняют Суть, Проблему, Решение, Ограничения; «согласен» даёт «+1», а не новую карточку.

Спринт 2 · 13:30–14:45 · Бот вмешивается и отвечает

Зависит от: спринт 1. Оценка: ~1 ч 15 мин. Самый рискованный спринт — если к 14:45 не сделан Impact, переносим его в спринт 4.

```
Задание для Claude — спринт 2.
1. triggers.py, guards.py, impact.py и операции Reducer (merge_nodes,
add_conflict,
   resolve_conflict, set_status, add_dependency, change_constraint,
resolve_question) —
   по старому техдоку, без правил про pinned. Юнит-тесты на guards и
impact.affected.
2. Промпты mediator.md и impact.md + MediatorResult / ImpactResult из старого
   техдока; в чате обращение по именам из Telegram. Ветки в pipeline.py.
3. Отправка chat_message в группу: реплаем на сообщение-причину, inline-
кнопка
   «Показать на канвасе» (ссылка /t/{token}#N7), cooldown BOT_COOLDOWN_S=60
для
   инициативных сообщений. Храни tg_message_id -> conflict_id: реплай на
вопрос
   бота = триггер RESOLVE. Реплай на другое сообщение бота = ask.
4. Answerer: промпт из техдока, быстрая модель, /no_idea ask <имя> <вопрос> и
реплай;
   работает и для закрытых тем. Вопросы на канвас не попадают.
5. На сайте: красное ребро конфликта и жёлтый пунктир needs_review (react-
xarrows),
   rejected серый зачёркнутый с тултипом, тосты, подсветка узла по #N7,
   индикатор «бот читает / думает».
Прогони replay demo_shop до реплики 8 и сверь с таблицей сценария; что не
совпало — перечисли.
```

Готово, когда: в ТГ реплика 5 даёт вопрос бота реплаем, реплай руководителя зачёркивает «Рассылку 20%», реплика 7 желтит карточки, ask отвечает по делу.

Спринт 3 · 14:45–15:45 · Итог темы, MVP-фриз

Зависит от: спринт 1 (канвас), Playwright на ноутбуке. Можно начать параллельно со спринтом 2 во второй сессии Claude. Оценка: ~1 ч.

```
Задание для Claude — спринт 3.
1. snapshot.py: один долгоживущий Chromium через Playwright, скриншот
   /t/{token}?print=1 (ширина 1600, без ленты и анимаций, все блоки
развёрнуты),
   ждать флаг window.__canvasReady; таймаут 10 с -> None.
2. Summarizer: промпт из техдока, сильная модель, саммари ⩽ 1000 символов +
ссылка.
3. Consolidator по старому техдоку (⩽ 6 операций, каждые 8 сообщений и перед
stop).
4. /no_idea stop: Consolidator -> тема closed -> Summarizer -> sendPhoto(PNG,
caption=саммари).
   Если PNG не получился — саммари текстом. На сайте плашка «Тема закрыта».
5. /no_idea export: PNG + sendDocument(export.md) + ссылка; для открытых и
закрытых тем.
   export.md — шаблоном без LLM по старому техдоку, сверху саммари, без
раздела «План».
6. start по имени закрытой темы переоткрывает её и снимает плашку.
Прогони весь demo_shop через replay, сохрани PNG и .md в data/ и покажи
саммари.
```

Готово, когда: в ТГ весь сценарий демо проходит от start до stop, PNG читается на телефоне. После 15:45 — только фиксы.

Спринт 4 · 15:45–17:00 · Надёжность и ТГ-тесты

Зависит от: спринт 3. Твоя часть — ручной чек-лист из раздела «Демо и тесты в Telegram» (~40 мин), баги сразу передаёшь Claude.

```
Задание для Claude — спринт 4.
1. scenarios/teammates.json и coffee.json из старого техдока; скрипт
run_scenarios.py
   прогоняет все три и проверяет чек-лист «Что проверяем во всех трёх»
автоматически
   там, где можно (дубли, слияние узлов конфликта, длина тезисов, ⩽ 6 на
блок).
   Исправь промпты по результатам.
2. Восстановление тем и ChatState из журналов при рестарте, без LLM.
3. safe() и деградация по старому техдоку, FALLBACK_MODEL, обработка ошибок
Telegram
   (RetryAfter, бот удалён из группы). Страница /debug/llm.
4. Полировка канваса для проектора: крупный шрифт, не «трясётся».
5. Фиксы из ручного теста в Telegram (список пришлю).
```

Готово, когда: все пункты ручного чек-листа зелёные, в 16:30 прошла репетиция в «no_idea demo» с CACHE_MODE=record и репликами копипастом.

Спринт 5 · 17:00–защита · Страховка и питч

- [ ] 17:00 — записать запасное видео: ТГ и канвас рядом, от start до stop
- [ ] Код-фриз, `CACHE_MODE=replay`, debounce как на репетиции, тег релиза
- [ ] Питч (можно попросить Claude собрать слайды): «Второй мозг в чате, где команда уже сидит»; «агент физически не может склеить конфликт — это гарантирует код»; клик по тезису как ответ на «а если ИИ наврал?»; /debug/llm вживую
- [ ] Один полный прогон питча + демо в replay
- [ ] Перед выходом: точка доступа включена, туннель жив, тема «Продажи» в demo-группе сброшена

Что резать, если отстаём

1. Восстановление из журнала при рестарте — просто не перезапускаем сервер на защите
2. Тестовый чат coffee.json
3. Consolidator по счётчику — только перед stop
4. Impact-анализ — из демо выпадает реплика 7
5. Рёбра SVG — красные бейджи «конфликт с N2» на карточках

Не режем: мгновенный приём сообщений из ТГ, вопрос бота на конфликт, guards, PNG + саммари на stop, replay-кэш и видео.